这是《CLR via C#》，你先熟悉一下： https: / / github.com / fhlsteven / CLR_via_CSharp / blob / main / chapters / ch9_Parameters.md

用户:

这是《CLR via C#》，你先熟悉一下：

https://github.com/fhlsteven/CLR_via_CSharp/blob/main/chapters/ch9_Parameters.md

元宝:

我已经熟悉了《CLR via C#》第9章“参数”的内容。这一章详细讲解了C#中参数的各种特性和使用方式，包括：

- **可选参数和命名参数**：如何为参数指定默认值，以及调用时如何通过参数名传递实参，还涉及了 `DefaultParameterValueAttribute` 和 `OptionalAttribute` 特性。
- **隐式类型的局部变量**：使用 `var` 关键字让编译器推断局部变量类型，以及它与 `dynamic` 的区别。
- **以传引用的方式传递参数**：通过 `out` 和 `ref` 关键字实现引用传递，包括它们的行为差异和适用场景。
- **向方法传递可变数量的参数**：使用 `params` 关键字定义接受可变数量参数的方法，以及相关的性能考虑。
- **参数和返回类型的设计规范**：建议参数类型尽量“弱”（如使用接口），返回类型尽量“强”，以提升代码的灵活性和可维护性。
- **常量性**：解释了CLR为何没有像C++那样提供常量性支持，并提到了不可变类型（如 `String` ）的设计思路。

如果你对其中任何部分有具体问题，或者想深入讨论某个主题，可以随时告诉我。

用户:

这部分不太懂，你再解释一下：

如果方法从模块外部调用，更改参数的默认值具有潜在的危险性。call site①在它的调用中嵌入默认值。如果以后更改了参数的默认值，但没有重新编译包含 call site 的代码，它在调用你的方法时就会传递旧的默认值。可考虑将默认值0/null作为哨兵值使用，从而指出默认行为。这样一来，即使更改了默认值，也不必重新编译包含了 call site 的全部代码。下面是一个例子：

元宝:

这部分内容描述的是C#可选参数的一个**重要设计隐患和解决方案**，核心是**版本控制**和**二进制兼容性**问题。我来为您详细解释一下。

核心问题：默认值被“固化”在调用方

当您为一个方法的参数设置默认值（如 `string filename = "Untitled"` ）时，这个默认值的行为在编译时就确定了，并且会“固化”在**每一个调用该方法的地方**。

1. **编译时（假设默认值为 `"Untitled"`）**：
   - 您编写了一个库，其中包含方法 `MakePath(string filename = "Untitled")`。
   - 其他程序员（调用方）引用您的库，并写代码调用 `MakePath()`。
   - **关键点**：C#编译器在编译调用方的代码时，发现 `filename` 参数缺失，它会**直接将默认值 `"Untitled"` 作为字面量，嵌入到调用方的程序集（DLL/EXE）中**。生成的代码等价于 `MakePath("Untitled")`。这个“调用地方”就是文档中提到的 **call site**。
2. **发布新版本时（您将默认值改为 `"Default.txt"`）**：
   - 您更新了库，将方法改为 `MakePath(string filename = "Default.txt")`，并发布新版本的DLL。
   - 调用方**没有重新编译他们的程序**，只是替换了您的新版DLL。
   - **问题发生**：当调用方程序运行时，它执行的代码仍然是当初编译时嵌入的 `MakePath("Untitled")`。它**不会**自动去读取您新DLL中定义的 `"Default.txt"`。因此，即使您更改了源码中的默认值，旧版本的调用方代码依然传递着旧的默认值。

这就导致了**潜在的危险性**：您看似修改了默认行为，但实际上，所有未重新编译的现有客户端程序的行为都没有改变，造成了版本间的行为不一致。

解决方案：使用哨兵值

文档建议的解决方案是**避免将“有业务意义的默认值”直接写在参数默认值里**，而是用一个无意义的“哨兵值”（如 `null`, `0`, `Guid.Empty` 等）作为参数默认，然后在**方法内部**实现真正的默认逻辑。

对比分析文档中的例子：

❌ **不好的做法（直接硬编码默认值）：**

```
private static String MakePath(String filename = "Untitled") {
    return String.Format(@"C:\{0}.txt", filename);
}
```

- **风险**：如上所述，更改 `"Untitled"` 会破坏未重新编译的客户端。

✅ **好的做法（使用哨兵值 `null`）：**

```
private static String MakePath(String filename = null) {
    // 如果 filename 是哨兵值 null，则使用真正的默认值 "Untitled"
    return String.Format(@"C:\{0}.txt", filename ?? "Untitled");
}
```

这个方案为何有效？

1. **稳定的合约**：方法的公开签名（接口）是稳定的，默认值始终是 `null`。无论您将来想把真正的默认文件名改成 `"Default.txt"` 还是 `"NewFile.txt"`，参数默认值 `null` 永远不会变。
2. **行为在内部实现**：真正的默认逻辑（`?? "Untitled"`）是方法**内部实现**的一部分。调用方编译时嵌入的固定值是 `null`。
3. **安全地更改行为**：当您想改变默认文件名时，只需修改方法内部的实现（例如将 `?? "Untitled"` 改为 `?? "Default.txt"`），然后发布新的DLL。

4. **保证兼容性**：所有调用方（无论新旧）传递的都是 `null`。旧版本客户端调用该方法，会立刻使用您**新DLL内部的新逻辑**（生成 `"Default.txt"`），因为它们传递的 `null` 这个“信号”没有变，变的是方法内部响应这个信号的逻辑。**因此，调用方无需重新编译。**

总结：
这个技巧的本质是将**编译时决定的默认值**，转变为**运行时决定的默认行为**。它通过一个固定的、无意义的**哨兵值**作为参数默认，将易变的业务逻辑隐藏在方法内部，从而实现了二进制层面的向后兼容，是库（API）设计中的一个重要最佳实践。

用户:

这块不太懂：

如果参数用ref或out关键字进行了标识，就不能设置默认值。因为没有办法为这些参数传递有意义的默认值。

使用可选或命名参数调用方法时，还要注意以下附加的规则和原则。

实参可按任意顺序传递，但命名实参只能出现在实参列表的尾部。

可按名称将实参传给没有默认值的参数，但所有必须的实参都必须传递(无论按位置还是按名称)，编译器才能编译代码。

C# 不允许省略逗号之间的实参，比如 M(1, , DateTime.Now)。因为这会造成对可读性的影响，程序员将被迫去数逗号。对于有默认值的参数，如果想省略它们的实参，以传参数名的方式传递实参即可。

如果参数要求 ref/out，为了以传参数名的方式传递实参，请使用下面这样的语法：

元宝:

这部分内容讲的是使用**可选参数、命名参数**以及结合 **ref/out 参数**时的具体语法规则。我来为您逐一解释：

---

1. 为什么 ref/out 参数不能有默认值？

文档解释：因为没有办法为这些参数传递有意义的默认值。

深入理解：

- `ref` 和 `out` 参数需要的是**变量的引用**（内存地址），而不是一个普通的值。
- 默认值通常是一个**编译时常量**（如 `0`、`null`、`"A"` 等），这些常量没有存储位置（地址）。
- 如果允许设置默认值，比如 `ref int x = 5`，那么调用时 `M(ref 5)` 是无效的，因为字面量 `5` 没有内存地址。
- 同理，`out` 参数要求在方法内部赋值，如果设置默认值，就违背了 `out` 参数“必须在方法内赋值”的语义。

示例：

```
// ❌ 这是非法的，编译错误
void Process(ref int value = 10) { ... }
void GetResult(out string data = null) { ... }  // 同样错误
```

---

2. 可选/命名参数的调用规则

规则一：命名实参必须放在位置实参之后

```
void Print(string name, int age = 20, string city = "Unknown") { ... }

// ✅ 正确：位置参数在前，命名参数在后
Print("Alice", city: "New York");  // name="Alice", age=20, city="New York"

// ❌ 错误：命名参数在前，位置参数在后
Print(city: "New York", "Alice");  // 编译错误！
```

规则二：必须传递所有必需的参数

```
void Calculate(int required1, int required2, int optional = 0) { ... }

// ✅ 正确：通过位置传递必需参数
Calculate(1, 2);

// ✅ 正确：通过命名传递必需参数
Calculate(required1: 1, required2: 2);

// ❌ 错误：缺少必需参数
Calculate(required2: 2);  // 缺少 required1
```

规则三：不能用空逗号跳过参数

```
void M(int a = 1, int b = 2, int c = 3) { ... }

// ❌ 错误：试图跳过中间参数
M(1, , 3);  // 编译错误！程序员需要数逗号位置，容易出错

// ✅ 正确：用命名参数跳过中间参数
M(1, c: 3);  // a=1, b=2(默认), c=3
```

---

3. 为 ref/out 参数使用命名参数的特殊语法

这是最特殊的一点：当使用命名参数调用有 `ref` / `out` 参数的方法时，需要在**参数名前**加上 `ref` / `out` 关键字。

文档示例：

```
// 方法声明
private static void M(ref Int32 x) { ... }

// 方法调用：
```

```
Int32 a = 5;
M(x: ref a);  // ✅ 注意语法：x: ref a
```

错误写法对比：

```
M(ref x: a);  // ❌ 错误：ref 应该在参数名和冒号之后
M(ref a);     // ✅ 正确：传统写法，但这不是命名参数
M(a);         // ❌ 错误：缺少 ref 关键字
```

更复杂的示例：

```
void Process(ref int x, string name, out bool success) {
    success = true;
}

int num = 10;
bool result;

// 混合使用位置参数、命名参数、ref/out参数
Process(ref num, "test", success: out result);  // ✅ 位置参数 + 命名out参数

// 全部使用命名参数
Process(x: ref num, name: "test", success: out result);  // ✅
```

---

4. 为什么需要这种特殊语法？

1. **明确性**：在调用点明确标出 `ref` / `out`，让代码阅读者清楚知道这个参数是按引用传递的。
2. **一致性**：保持与普通参数调用语法的一致性（ `参数名: 值` ），只是值部分变成了 `ref 变量` 或 `out 变量`。
3. **编译器需求**：帮助编译器区分普通值和引用传递。

总结要点

场景	正确语法	错误语法
调用有默认值的参数	`M(1, c: 3)`	`M(1, , 3)`
必需参数传递	`M(1, 2)` 或 `M(a: 1, b: 2)`	`M(b: 2)`
ref参数+命名参数	`M(x: ref a)`	`M(ref x: a)`
out参数+命名参数	`M(result: out data)`	`M(out result: data)`

关键记忆：

- 想跳过有默认值的参数 → 用命名参数
- 调用有 `ref` / `out` 的方法 → 在命名参数语法中，`ref` / `out` 关键字放在变量前
- 命名参数必须放在所有位置参数之后

用户:

这里不太懂：

重要提示 经常有人问我，为什么 C# 要求必须在调用方法时指定 out或ref？毕竟，编译器知道被调用的方法需要的是out还是ref，所以应该能正确编译代码。事实上，C# 编译器确实能自动采用正确的操作。但 C# 语言的设计者认为调用者的方法是否需要对传递的变量值进行更改。

元宝:

这部分内容解释了C#语言设计中一个**重要的设计决策**。让我详细解释为什么C#要求在调用方法时必须显式指定 `out` 或 `ref` 关键字。

核心原因：明确表达调用者的意图

文档中这段话的核心观点是：虽然**编译器完全有能力自动识别**方法需要 `ref` 还是 `out` 参数，但C#语言设计者**故意**要求调用者必须显式指定，这是为了**提高代码的可读性和安全性**。

实际例子说明

假设有以下方法定义：

```
// 方法定义
void UpdateValue(ref int data) {
    data = data * 2;
}

void GetResult(out int result) {
    result = 42;
}
```

情况一：如果C#不要求显式指定（假设的语法）

```
int x = 10;
UpdateValue(x);  // ❌ 假设的语法：不写ref
Console.WriteLine(x);  // x会被修改吗？阅读者不知道！
```

问题：看到 `UpdateValue(x)` 这行代码，你无法立即知道：

- `x` 的值是否会被方法修改？
- 你需不需要在调用前初始化 `x` ？
- 调用后 `x` 的值是多少？

情况二：C#实际的要求

```
int x = 10;
UpdateValue(ref x);  // ✅ 实际语法：必须写ref
Console.WriteLine(x);  // 输出20
```

优势：看到`ref x`，你立即知道：

1. `x`的值**可能**会被方法修改
2. 你必须在调用前初始化`x`
3. 调用后应该检查`x`的新值

为什么这是好的设计？

1. 代码可读性

`ref`/`out`关键字是**调用者的承诺标志**：

- `ref`承诺："我知道这个变量可能会被修改"
- `out`承诺："我知道这个方法会给我一个新值"

2. 避免意外修改

想象一下如果没有这个要求：

```
int importantValue = 100;
SomeMethod(importantValue);  // 这个方法会修改importantValue吗？
// ... 很多行代码后 ...
Console.WriteLine(importantValue);  // 还是100吗？
```

你必须查看`SomeMethod`的定义才能知道它是否会修改参数。有了`ref`关键字，这个问题一目了然。

3. 与值传递形成明确对比

```
// 值传递 - 原始值不会被修改
int a = 5;
ProcessByValue(a);  // a仍然是5

// 引用传递 - 原始值会被修改
int b = 5;
ProcessByRef(ref b);  // b会被修改，阅读者看到ref就知道这一点
```

文档中提到的"编译器确实能自动采用正确的操作"

文档这句话的意思是：**从技术上讲，编译器确实知道**：

- 查看方法签名就知道参数需要`ref`还是`out`

- 可以自动在调用点插入正确的代码，不需要程序员写 `ref` / `out`

但C#设计者**有意不这样做**，因为：

```
// 如果编译器自动处理，可能会发生这种情况：
BankAccount account = GetMyAccount();
account.Withdraw(100);  // 看起来只是查询余额？
// 实际上Withdraw方法修改了账户余额，但调用点没有提示！
```

类比：其他语言的不同选择

为了更好理解，对比其他语言：

语言	参数传递方式	是否显式标记
C#	`ref` / `out`	**必须**在调用点显式写 `ref` / `out`
C++	`&` 引用参数	只在声明时需要 `&`，调用时不特殊标记
Java	只有值传递	无法直接修改原始变量（通过对象引用间接修改）

C#选择了最**明确**的方式，虽然写起来稍微麻烦，但大大减少了bug。

实际开发中的价值

1. **代码审查**：审查者一眼就能看出哪些调用可能修改状态
2. **调试**：看到 `ref` 就知道要检查这个变量的前后变化
3. **团队协作**：新人阅读代码时，不需要跳转到每个方法定义去查看参数传递方式

总结

C#要求显式指定 `ref` / `out` 不是技术限制，而是**有意的设计选择**，目的是：

1. **提高代码可读性** - 调用点明确表达"这里会修改变量"
2. **增强代码安全性** - 避免无意中修改变量
3. **明确程序员意图** - 看到 `ref` 就知道你是有意要修改变量
4. **便于维护** - 不需要查看方法定义就知道参数传递方式

这是一个典型的"让正确的事情显而易见，让错误的事情难以发生"的设计哲学体现。虽然多写几个字符，但带来的代码清晰度和可维护性的提升是非常值得的。

用户:

这块不太懂，你结合例子解释一下：

9.6 常量性

有的语言(比如非托管 C++)允许将方法或参数声明为常量，从而禁止实例方法中的代码更改对象的任何字段，或者更改传给方法的任何对象。CLR 没有提供这个功能，许多程序员因此觉得很遗憾。既然 CLR 都不提供，面向它的任何编程语言(包括 C#)自然也无法提供。

首先要注意，非托管 C++ 将实例方法或参数声明为 const 只能防止程序员用一般的代码来更改对象或参数。方式内部总是可以更改对象或实参的。这要么是通过强制类型转换来去掉“常量性”，要么通过获取对象/实参的地址，再向那个地址写入。从某种意义上说，非托管 C++ 向程序员撒了一个谎，使他们以为常量对象或实参不能写入(事实上可以)。

实现类型时，开发人员可以避免写操纵对象或实参的代码。例如，String 类就没有提供任何能更改 String 对象的方法，所以字符串是不可变(immutable)的。

此外，Microsoft 很难为 CLR 赋予验证常量对象/实参未被更改的能力。CLR 将不得不对每个写入操作进行验证，确定该写入针对的不是常量对象。这对性能影响很大。当然，如果检测到有违反常量性的地方，会造成 CLR 抛出异常。此外，如果支持常量性，还会给开发人员带来大量复杂性。除此之外，在不可变的类型中，字段也必须不可变。

考虑到这些原因以及其他许多原因，CLR 没有提供对常量对象/实参的支持。

元宝:

文档这部分解释了为什么 CLR（以及 C#）**没有**像 C++ 那样提供语言级的 `const` 关键字来保证“常量性”。我来结合例子，为您详细拆解。

---

1. C++ 的 `const` 是如何工作的？（以及它如何“撒谎”）

在 C++ 中，你可以：

1. 将**方法**声明为 `const`，承诺该方法不会修改对象的成员。
2. 将**参数**声明为 `const &`，承诺不会通过这个引用修改传入的对象。

C++ 示例：

```
class MyObject {
private:
    int value;
public:
    // 一个 const 方法，承诺不修改对象
    int getValue() const {
        return value; // 正确：只是读取
        // value = 10; // 错误：在 const 方法中不能修改成员
    }

    void mutate() {
        value = 20; // 正确：非 const 方法可以修改
    }
};

// 一个接收常量引用的函数，承诺不修改传入对象
void printValue(const MyObject& obj) {
    cout << obj.getValue(); // 可以调用 const 方法
```

```
        // obj.mutate(); // 错误：不能通过 const 引用调用非 const 方法
    }
```

但是，C++ 的“常量性”是可以被绕过的：

```
    class Deceptive {
    private:
        int secret = 0;
    public:
        int getSecret() const {
            // 通过“强制类型转换去掉常量性”来修改！
            const_cast<Deceptive*>(this)->secret = 42; // 🧯 编译通过，运行时会修改！
            return secret;
        }
    };
```

文档所说的“撒谎”：

- C++ 的 `const` 是一个**编译时承诺**，而非运行时强制保护。
- 编译器会阻止“常规代码”的修改，但开发者可以通过 `const_cast` 或获取地址后写入等“非常规”手段，在 `const` 方法内部或通过 `const` 引用修改对象。
- 因此，C++ 让程序员“以为”对象不能被修改，但实际上**技术上**仍然可以，这降低了其保证的可信度。

---

2. 为什么 CLR 不提供这个功能？

如果 CLR 要实现**真正可靠**的常量性，就需要解决以下难题，而文档认为这些代价太高：

a) 性能开销巨大

如果 CLR 要确保一个被声明为常量的对象在运行时不被修改，它必须：

- 在**每一次**可能写入内存的操作（如设置字段、调用可能修改对象的方法）前进行检查。
- 跟踪整个对象图（比如对象内部的嵌套对象）的常量状态。
  这种持续的运行时检查会严重拖慢程序执行速度。

b) 实现极其复杂

- 如何定义“常量”的边界？一个常量对象的字段如果引用另一个对象，那个对象是否也是常量？
- 如何处理反射？`const` 对象通过反射还能修改吗？
- 如何处理与非托管代码的互操作？非托管代码可不管 CLR 的常量性。

c) 对类型设计的限制过强

如果支持真正的常量性，那么一个“不可变类型”必须满足：

- 所有字段都必须是只读的。
- 字段的类型本身也必须是不可变的。
  这在实际编程中限制太大，很多有用的模式无法实现。

---

3. C# 如何应对？——“不可变性”代替“常量性”

既然 CLR 不提供底层支持，C# 采取了另一条路：**通过类型设计和语言特性，在源码层面鼓励和实现“不可变性”**，而不是依赖一个可能被绕过的 `const` 修饰符。

文档中的例子：`String` 类

`String` 是“不可变”的经典例子。它**没有提供任何可以修改当前实例内容的方法。**所有看似修改的方法（如 `Replace`，`ToUpper`）都**返回一个全新的字符串对象。**

```
string s1 = "Hello";
string s2 = s1.ToUpper(); // s1 仍然是 "Hello"，s2 是新的 "HELLO" 对象
// s1[0] = 'h'; // 错误：不允许。索引器只有 get 访问器。
```

这种不可变性是**由类的设计者（Microsoft）在编写 `String` 类时主动保证的**，而不是由 CLR 通过 `const` 关键字强制保证的。

C# 中实现不可变性的现代工具：

1. **`readonly` 字段**：确保字段只能在构造函数中赋值。
2. **Init-only 属性（`init` 访问器）**：对象初始化后属性就不能再变。
3. **记录类型（`record`）**：默认就是不可变的。
4. **不可变集合**（`System.Collections.Immutable` 命名空间）。

示例：在 C# 中创建一个不可变类型

```
public class Person
{
    // 1. 字段设置为 readonly
    public readonly string Name;
    public readonly int Age;

    // 2. 通过构造函数提供初始值
    public Person(string name, int age)
    {
        this.Name = name;
        this.Age = age;
    }

    // 3. 任何“修改”操作都返回一个新实例
    public Person WithAge(int newAge)
    {
        return new Person(this.Name, newAge);
    }
}

// 使用
```

```
    var person = new Person("Alice", 30);
    var olderPerson = person.WithAge(31); // person 保持不变，得到一个新对象
```

总结对比

特性	C++ `const`	C# 的应对策略
机制	语言关键字，**编译时检查。**	**类型设计模式**，由开发者保证。
可靠性	可被 `const_cast` 等绕过，是“绅士协议”。	如果正确使用 `readonly` 等，是**真正不可变**的（运行时也改不了）。
性能	零运行时开销。	零运行时开销（因为不依赖 CLR 验证）。
灵活性	可以修饰方法、参数，粒度细。	作用于整个类型或成员，通过返回新对象实现“修改”。
CLR/运行时支持	不涉及。	**CLR 不提供原生支持**，这是 C# 语言和框架层面的设计。

核心结论：
文档解释了 CLR 团队的一个**权衡决策**：不引入一个复杂、有性能代价、且可能被绕过的“常量性”系统，而是将“保证对象不被修改”的责任交给**类型的设计者**和**开发者遵循的编程模式。**C# 通过提供 `readonly`、`record` 等特性，让创建不可变类型变得更容易，以此来实现类似（甚至更可靠）的目标。

用户:

总结一下第九章

元宝:

根据《CLR via C#》第九章的内容，本章系统地讲解了C#中方法参数的核心机制、设计模式及其底层原理。以下是本章各节的重点总结：

主题	核心要点	关键细节/注意事项
1. 可选参数与命名参数	允许为参数设置默认值，调用时可省略或按名称指定。	- 默认值必须是编译时常量。 - **版本控制风险**：默认值在调用点编译时被固化，修改默认值后，未重新编译的调用方仍使用旧值。 - 建议用 `null` / `0` 作“哨兵值”，在方法内部实现默认逻辑，以保持二进制兼容性。
2. 隐式类型局部变量（`var`）	编译器根据初始化表达式推断局部变量类型。	- 只能用于局部变量，不能用于字段、参数或返回类型。 - 常用于匿名类型、复杂泛型实例（如 `Dictionary<string, int>`），简化代码。 - 区别于 `dynamic`：`var` 是编译时静态类型推断，`dynamic` 是运行时动态类型。
3. 引用传递（`ref` / `out`）	通过引用传递变量，允许方法直接修改调用方的变量。	- `ref`：调用前必须初始化，方法内部可读可写。 - `out`：调用前无需初始化，方法内部必须赋值。 - **设计意图**：C# 强制在调用点显式写 `ref` / `out`，是为了明确表达“该方法可能修改变量”，提高代码可读性和安全性。 - 引用类型使用 `ref` / `out` 通常用于“换引用”（让参数指向新对象）。
4. 可变数量参数（`params`）	使用 `params` 关键字定义接受可变数量参数的方法。	- 必须是方法的最后一个参数，且为一维数组。 - 编译器自动将实参包装成数组。 - **性能考虑**：频繁调用可能因数组分配影响性能，可为常见参数数量提供重载版本（如 `String.Concat` 的做法）。
5. 参数与返回类型设计规范	设计良好的API时应遵循的强弱类型原则。	- **参数类型尽量“弱”**：使用接口（如 `IEnumerable<T>`）或基类（如 `Stream`），提高方法通用性。 - **返回类型尽量“强”**：返回具体类型（如 `FileStream`），给予调用方更多操作自由，但若考虑未来变更，可返回较弱类型（如 `IList<T>`）。
6. 常量性	解释为何 CLR 不提供类似 C++ `const` 的语言级常量性支持。	- C++ 的 `const` 仅为编译时承诺，可通过强制转型绕过，非真正不可变。 - 实现真正的运行时常量性验证会带来巨大性能开销和复杂性。 - C# 通过**不可变类型设计**（如 `String`、`readonly` 字段、`record` 类型）在语言层面鼓励不可变性，而非依赖运行时强制约束。

本章核心思想贯穿线：C# 的参数设计始终在**灵活性**、**安全性**和**性能**之间寻求平衡。无论是可选参数的版本控制隐患、`ref` / `out` 的显式要求，还是放弃 `const` 而采用不可变模式，都体现了其“让常见操作简单、让危险操作显式、

让错误难以发生”的设计哲学。

（注：文档部分内容可能由 AI 生成）